PREORDER

Root · data
LST
RST

TC ⇒ O(N)

SC ⇒ ⟷

```
void preOrder(TreeNode root) {
    if(root == null)
        return
    print(root.data)
    preOrder(root.left)
    preOrder(root.right)
}
```

INORDER

LST
root · data
RST

TC ⇒ O(N)

```
void inOrder(TreeNode root) {
    if(root == null)
        return
    inOrder(root.left)
    print(root.data)
    inOrder(root.right)
}
```

POSTORDER

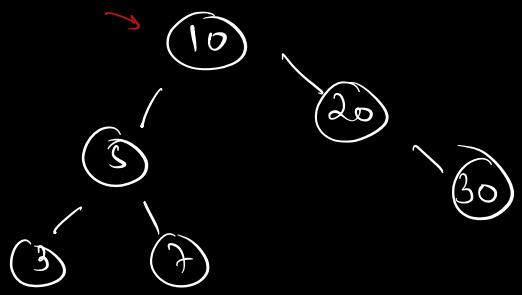
LST
RST
root · data

TC ⇒ O(N)

```
void postOrder(TreeNode root) {
    if(root == null)
        return
    postOrder(root.left)
    postOrder(root.right)
    print(root.data)
}
```

POSTORDER

```
void postOrder( treeNode root) {
    if( root == null)
        return
    postOrder( root.left)
    postOrder( root.right)
    print ( root.data)
}
```



O/p => 3 7 5 30 20 10

[LST
RST
root.data]

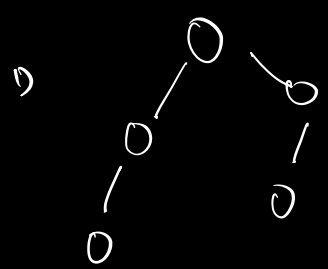
Google

[How to write the traversals if OS doesn't support recursion
↳ Stacks]

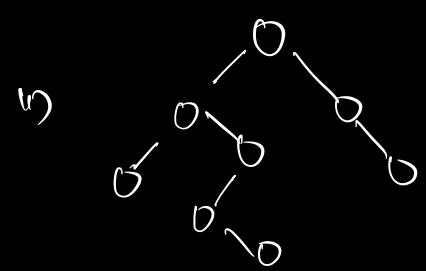
Q 1. Given a binary tree. find its height

↳ Length of longest path from root to any leaf node.

[no. of nodes]



h = 3.



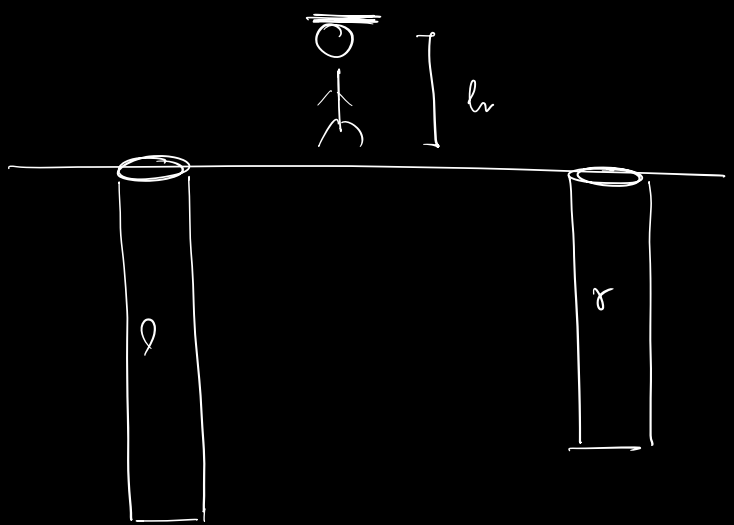
h = 5

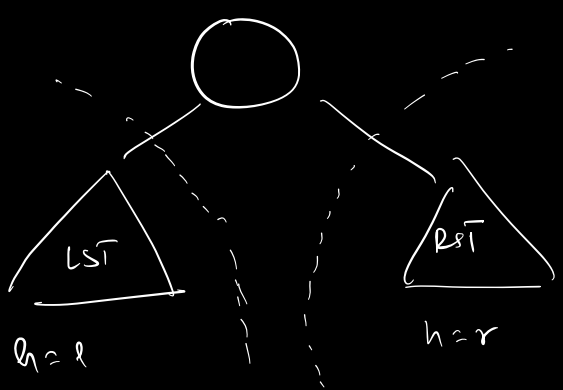
1111) ○ h = 1.

2) root == null h = 0.

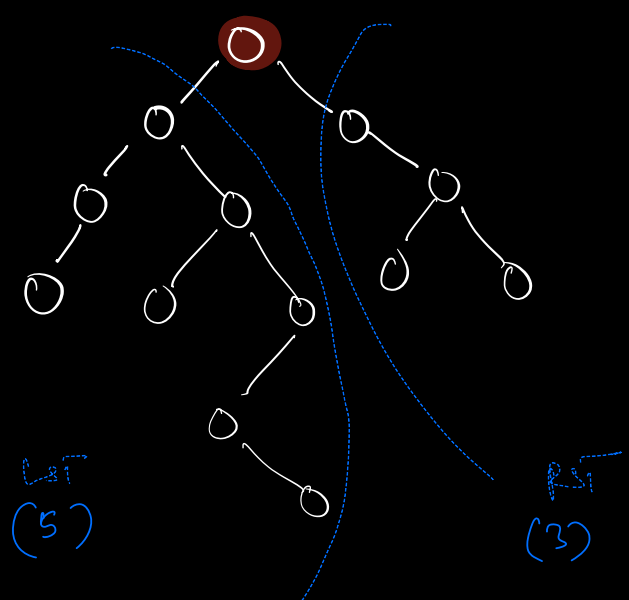
maximum depth

= max(l, r) + h





H = max(l, r) + 1

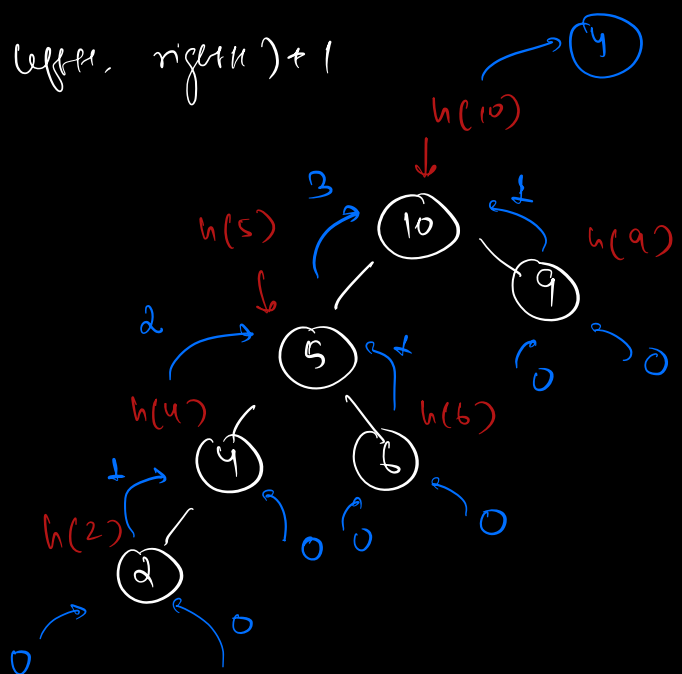


h = 6.

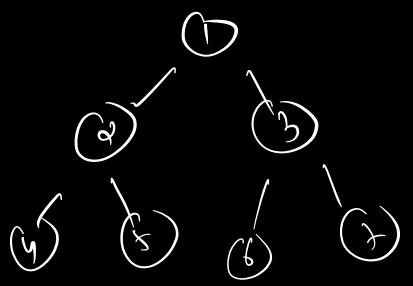
H = max(height(LST), height(RST)) + 1.

pseudo

```
int height(TreeNode root) {
    if(root == null)
        return 0;

    int lefth = height(root.left)
    int righth = height(root.right)

    return max(lefth, righth) + 1
}
```



h = 4

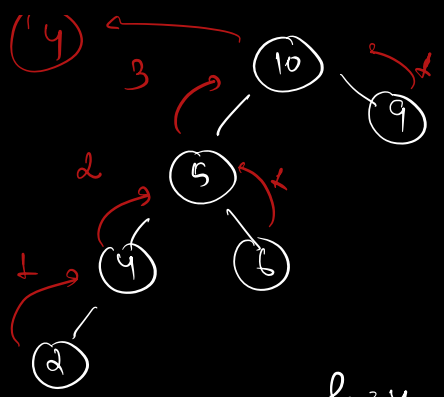


pre ⇒ 1 2 4 5 3 6 7

In ⇒ 4 2 5 1 6 3 7

post ⇒ 4 5 2 6 7 3 1

```
int height(TreeNode root) {
    if(root == null)
        return 0;
    int lefth = height(root.left)
    int righth = height(root.right)
    return max(lefth, righth)+1
}
```



h = 4

h(10)

lH = 3

rH = 1

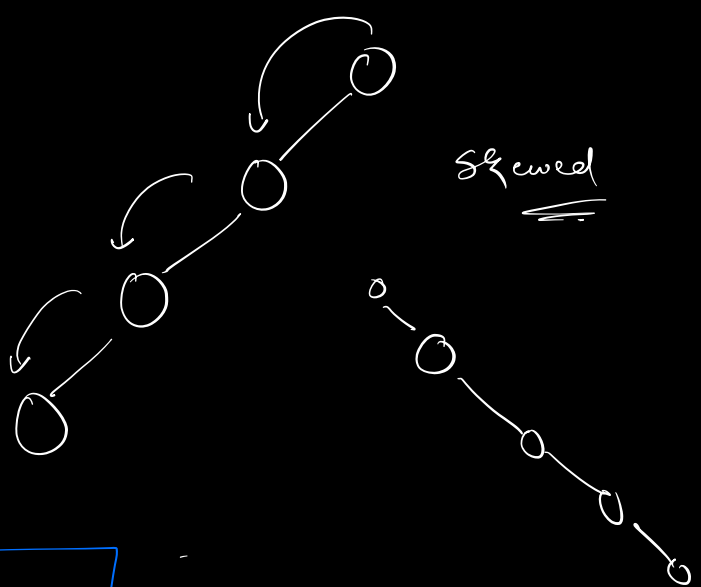
max(3,1)+1

3+1 = 4

TC ⇒ O(N)

SC ⇒ O(N)

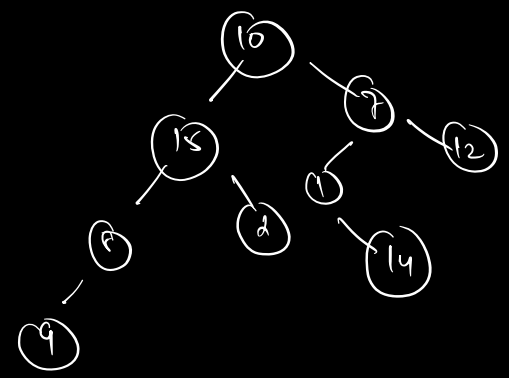
height ⇒ post order traversal



break ⇒ 7 mins ⇒ 8:10 PM IST

Q 2. Given a value 'k', check if no. exists in the tree.

eg ⇒



k = 2 → true

k = 20 → false

k = 10

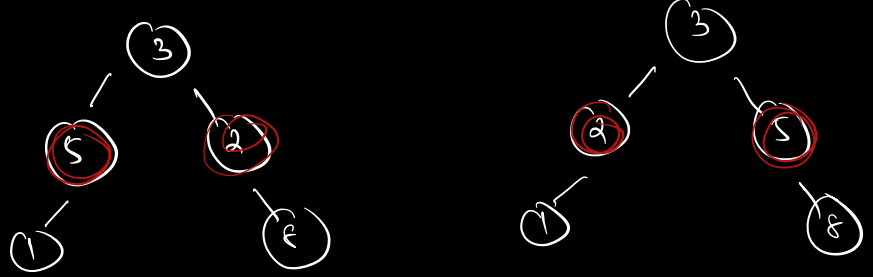
preorder ⇒ O(N) (worst)

postorder ⇒ O(N) ⟵ preferred ⇒ preorder

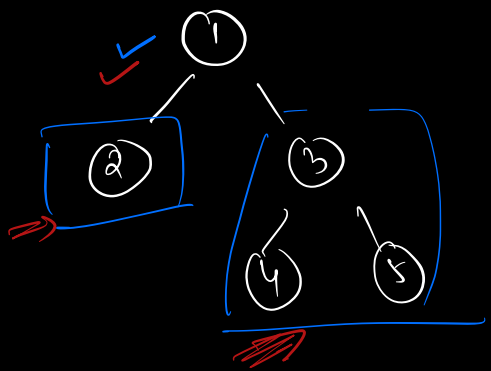
inorder ⇒ O(N)

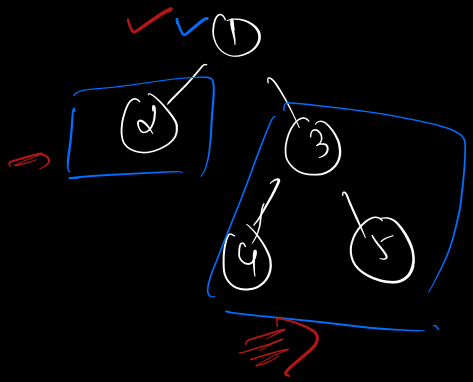
Q 3. Given two binary trees, check if they are identical.

eg ⇒

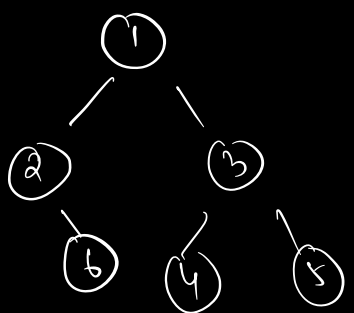


O/P ⇒ false





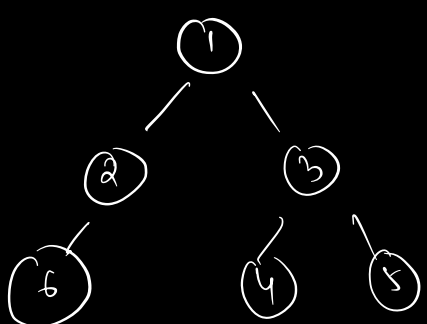
O/p ⇒ true.



pre ⇒ 1 2 6 3 4 5

In ⇒ 2 6 1 4 3 5

post ⇒ 6 2 4 5 3 1



pre ⇒ 1 2 6 3 4 5

In ⇒ 6 2 1 4 3 5

post ⇒ 6 2 4 5 3 1

Logic

1) find all 3 traversals ⇒ 3N + 3M.

t tree ⇒ N

t tree ⇒ M

2) compare all tree ⇒ 3 (min(N, M))

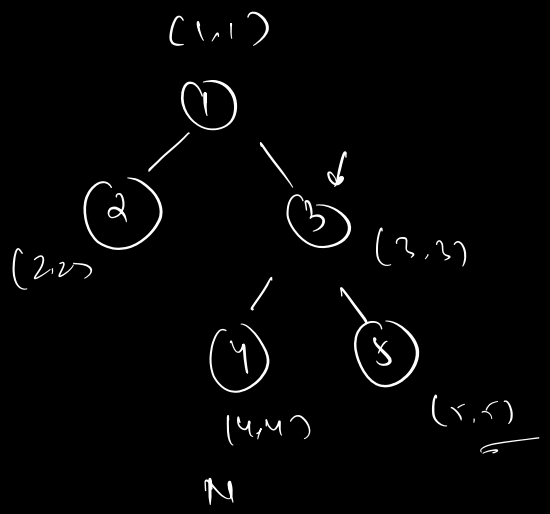
pseudo

```
boolean isIdentical(TreeNode root1, TreeNode root2) {

    if(root1 == null && root2 == null)
        return true;

    if(root1 == null || root2 == null)
        return false;

    if(root1.data != root2.data)
        return false;

    return [isIdentical(root1.left, root2.left)
    && isIdentical(root1.right, root2.right)]

}
```

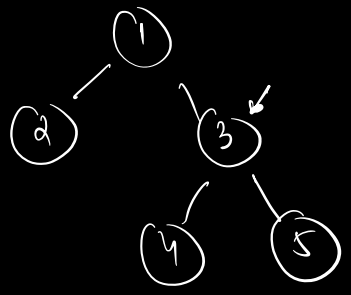
main logic ⇒

```
if( roots are equal )
&&
  if ( LST1 == LST2 )
&&
  if( RST1 == RST2 )
```

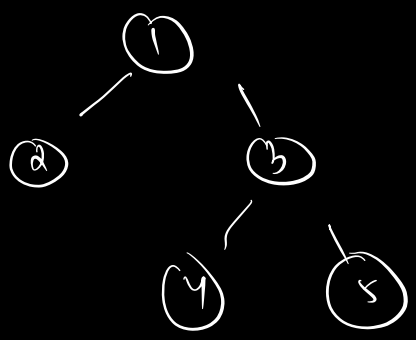
→ true

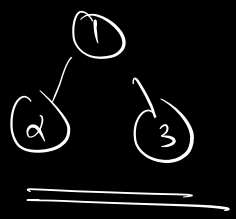


N



N





M

isEden(1,1)

isEden(2,2)

isEden(3,3)

isEden(4,null)

X

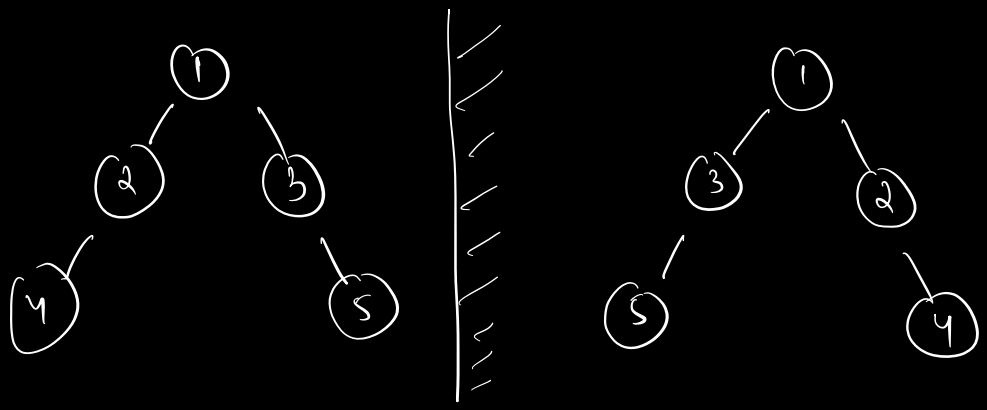
N

for

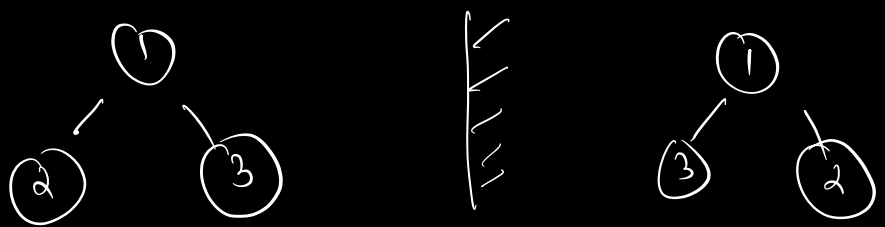
TC ⇒ O(min(N,M))

N & M are nodes at each tree respectively

Q 4. Given two binary trees, check if they are mirror image of one another.



O/p → true




```
if( root1.data == root2.data)
if( isIden( root1.left, root2.right)
  && isIden( root1.right, root2.left) )
```

pseudo

```
boolean isMirror ( TreeNode root1, TreeNode root2) {
    if( root1 == null && root2 == null)
        return true;

    if( root1 == null || root2 == null)
        return false;

    if(root1.data != root2.data)
        return false;

return [ isMirror ( root1.left, root2.right)
    && isMirror (root1.right, root2.left )]
}
```

⇒ Intro to BST

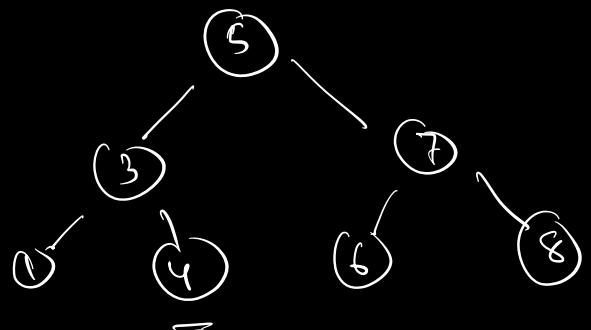
BST ⇒ binary search tree

↓ ↓

extension

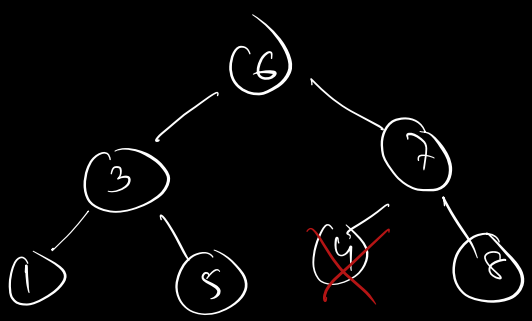
prop ⇒ [LST] ≤ root ≤ [RST]

all nodes.



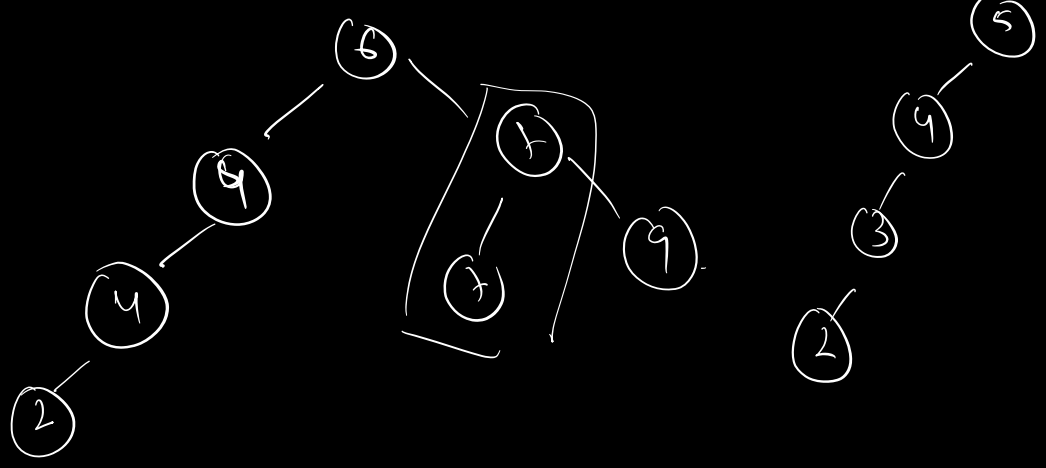
BST

$2 > 3$

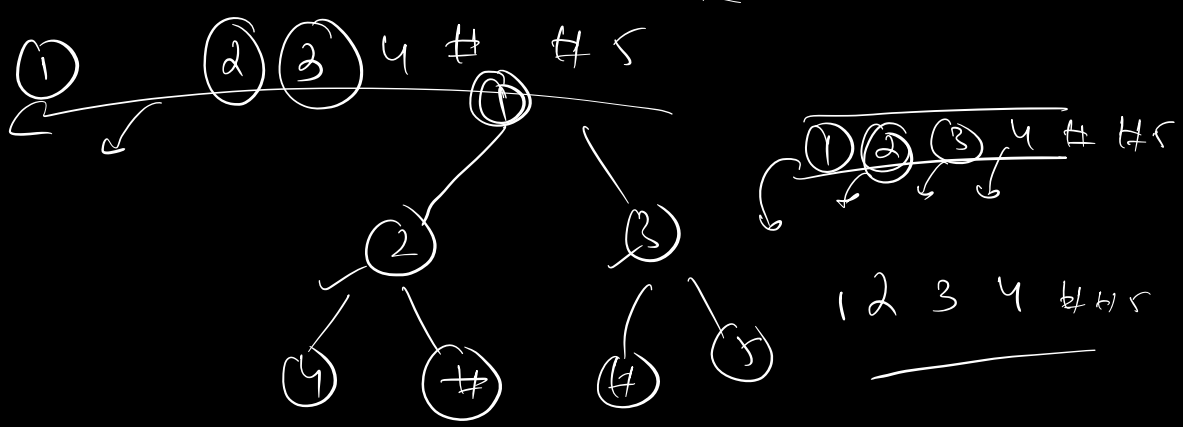


⇒ not a bst

$4 \leq 6$



lst $\leq$ root $\leq$ right


1 2 3 4 # # 5
1 2 3 4 # # 5
1 2 3 4 # # 5